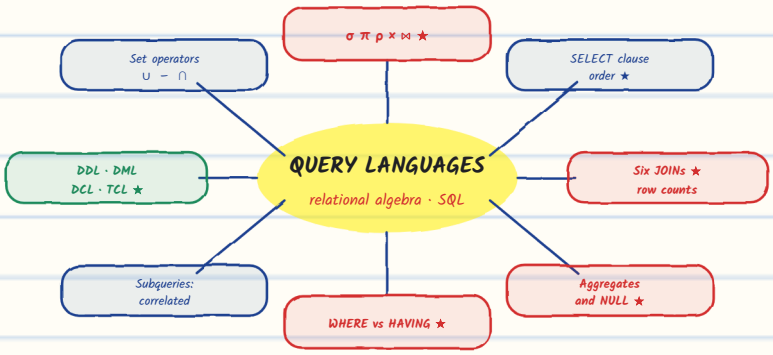
Query Languages: RA & SQL

Relational algebra · SQL command families · the six joins · aggregates, NULL & GROUP BY: exam-ready in 10 sheets



↑ the whole topic: red = high-yield, green = the practical side

➜ Syllabus coverage

The Database Systems syllabus reads: "Concepts of database systems; Data Models: Entity-Relationship model, Relational model; Query Languages: Relational algebra, SQL; Constraints and triggers; Functional dependencies and normalization; File organization and data storage; Indexing: primary and secondary indexes, B+ trees, hash tables; Query optimization; ..." **This lesson takes the "Query Languages" clause only.** Constraints and triggers are one line further down the same line — they were covered with the data models; normalisation, indexing and transactions have their own lessons.

High-yield list

① σ filters **rows**, π chooses **columns**
② ρ renames; × pairs every row with every row
③ DDL · DML · DCL · TCL — four families
④ **DROP** kills structure; DELETE/TRUNCATE do not
⑤ WHERE filters **before** grouping, HAVING **after**
⑥ **COUNT(*)=5, COUNT(a)=3** on 1,NULL,3,NULL,5
⑦ AVG of 10,NULL,20 = **15**, never 10
⑧ NULL never equals anything in a join
⑨ 6 joins: INNER, LEFT, RIGHT, FULL, CROSS, SELF
⑩ σ ⇒ the WHERE clause; π ⇒ the SELECT list

Legend yellow = key term · red circle = must-memorise · **purple** = memory trick · **red** = exam trap · **green** = worked example

➜ Contents

✓ RA: the eight operators ... p. 2
✓ Students / Courses worked in RA ... p. 3
✓ Product, rename, join, SQL ⇄ RA ... p. 4
✓ SQL: DDL, DML, DCL, TCL ... p. 5
✓ Aggregates, NULL, GROUP BY ... p. 6
✓ The six joins, row by row ... p. 7
✓ Join patterns & subqueries ... p. 8
✓ Night-before sheet ... p. 9
✓ Practice MCQs + written ... p. 10

1 Relational Algebra (RA): the Eight Operators

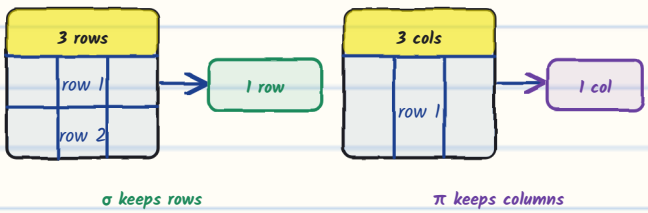
most tested ★

Definition **Relational algebra** is the theoretical language of queries: every request is written as a sequence of operations applied to relations, and the result of one step is the input of the next. It has five primitive operations (σ, π, ρ, ×, and the assignment) from which the rest — join, intersection, division — are derived. Every step returns a relation, so steps can be chained freely.

Operation	Symbol	What it does	Example
Selection	σ	keeps the rows that satisfy a condition	σ salary>50000 (Emp)
Projection	π	keeps the chosen columns; removes duplicates	π name,salary (Emp)
Union	∪	every tuple of both relations, once	A ∪ B
Set difference	−	tuples in A that are **not** in B	A − B
Intersection	∩	tuples present in both	A ∩ B
Cartesian product	×	**every row of A with every row of B**	Emp × Dept
Rename	ρ	renames a relation or an attribute	ρ pay/salary (Emp)
Join	⋈	product then selection — natural or θ	Emp ⋈ Dept

Derived operations Everything that is not primitive is built from the five: **θ-join** = $\sigma_{A=B}(R \times S)$ · **natural join** = θ-join plus automatic removal of the duplicated join attribute · **intersection** = A − (A − B) · **rename** is primitive, but ρ is what makes a self-join legal.

Worked example ✍ the set operators A = {1, 2, 3}, B = {3, 4} → A ∪ B = {1,2,3,4} (4, the 3 appears once) · A − B = {1,2} · A ∩ B = {3} · A × B = **3 × 2 = 6** tuples, all distinct. ∪, ∩ and − need **union-compatible** relations (same attributes, same domains) and never repeat a tuple. The product needs **no** common attribute name — which is why ρ comes before Students × Courses.



a query is a chain: one relation feeds the next

read it inside out

σ narrows downwards, π narrows sideways

Memory trick **"Sigma picks rows, pi picks columns."** σ is the letter you point at a line of a table with; π is used in a list. In SQL they are WHERE and SELECT — never the other way round.

Exam trap **σ does not change the number of columns** and **π does not change the number of rows** (it may remove duplicates). "Which operator drops a column?" → **π**. "Which operator drops a row?" → **σ**. And **ρ changes nothing at all**: only the label.

2 RA Worked on the Students / Courses Database

written favourite ★

Every RA question in an exam is asked about some concrete pair of relations. Learn these two and the symbol stops being abstract.

Students	id	name	dept
	1	Alice	CSE
	2	Bob	EEE
Courses	**cid**	**name**	
	C1	DBMS	
	C2	Algorithms	

RA query	Result
σ dept='CSE'(Students)	1 tuple: (1,Alice,CSE)
π name (Students)	2 tuples: Alice, Bob
π dept (Students)	2 tuples: CSE, EEE
Students × Courses	2 × 2 = 4 tuples

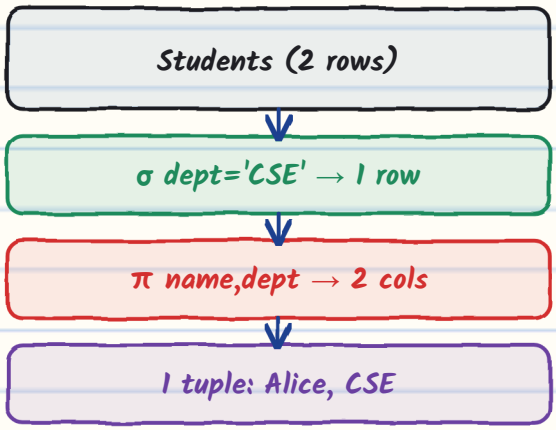
Exam trap The Cartesian product of these two relations is 4 tuples, not 2 and not 3: **rows × rows**. In general a product of *m* rows and *n* rows gives *m* × *n* rows — a 3-row and a 3-row relation give 9.

Worked example 1 ✍ "List the name and department of every CSE student."

RA: π name,dept (σ dept='CSE'(Students))

SQL:
```
SELECT name, dept FROM Students
WHERE dept='CSE';
```

Read the RA **inside out**: the innermost bracket is evaluated first, so σ filters and π then trims the columns.



inside-out evaluation

Worked example 2 ✍ "Show every student paired with every course."

RA: π name,cname (Students × ρ cname/name(Courses))

SQL:
```
SELECT s.name, c.name FROM
Students s, Courses c;
```

ρ is **mandatory**: after the product both relations have an attribute called *name*, and a relation may not carry two attributes of the same name.

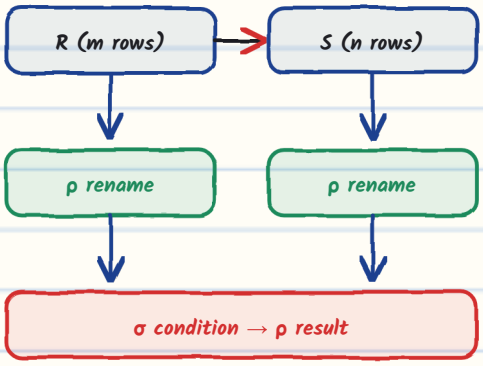
Exam trap ✍ the classical pair A paper offers σ name (Students) and π dept (Students). The first is a **type error**: *name* is an attribute, not a predicate, so σ needs a **condition** while π needs a **list of attributes**. And "π removes duplicates" is true of **RA** — plain SQL's SELECT keeps duplicates unless you write DISTINCT.

3 Product, Rename and Join — and Translating to SQL

join is the favourite ★

The join formula **θ-join** $R \bowtie_{A=\theta B} S \equiv \sigma_{A=\theta B}(R \times S)$ · ***natural join*** $R \bowtie S$ *joins on every attribute the two relations share* ***by name****, then drops the duplicate copy.*

Why ρ exists: *a relation may not carry two attributes of the same name, so* `Emp × Emp` *is illegal until one copy is renamed — which is also why* ***every self-join needs two aliases****.*



m × n rows, then only the pairs that pass

join = product + selection

Memory trick "***Product first, then pick.***" *That one sentence gives you the join, the semi-join and every θ-join you will be asked to expand. A* `WHERE`*-clause query looks like "join first, filter after" — the optimiser silently reorders it into exactly this shape.*

English question	Relational algebra	SQL
Employees paid over 50 000	`σ salary>50000 (Emp)`	`SELECT * FROM Emp WHERE salary > 50000`
Only names and salaries	`π name,salary (Emp)`	`SELECT name, salary FROM Emp`
All CSE students	`σ dept='CSE'(Students)`	`… WHERE dept='CSE'`
Salary renamed to pay	`ρ pay/salary (Emp)`	`SELECT salary AS pay`
Students of the CSE department	`Students ⋈ Dept`	`… JOIN Dept ON …`

Feature	Relational algebra	SQL
Nature	theoretical foundation	practical, what users and apps write
Style	procedural — say how	**declarative** — say only what
Duplicates	none (set theory); π removes them	kept unless `DISTINCT` is written
"Paid over 50 000"	`σ salary>50000 (Emp)`	`SELECT * FROM Emp WHERE salary>50000`
Optimisation	rewrites are exactly equal relations	the optimiser **converts SQL into an RA tree**, then picks the cheapest equivalent one

Exam trap *RA works on* ***sets****, SQL on* ***bags****, so the same question may return different row counts from the two languages — a deliberate, examinable difference, not a bug. And because SQL is declarative the user does not choose the order of operations: the optimiser does, by pushing selections down towards the leaves so the huge product never happens.*

4 SQL Command Types and the Order Clauses Run In

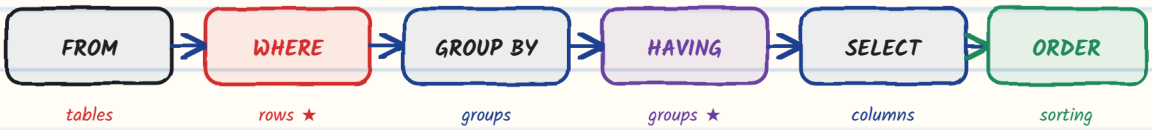
high-yield ★

Family	Full name	Commands	Acts on
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE	structure
DML	Data Manipulation Language	SELECT, INSERT, UPDATE, DELETE (also MERGE)	rows
DCL	Data Control Language	GRANT, REVOKE	privileges
TCL	Transaction Control Language	COMMIT, ROLLBACK, SAVEPOINT	transactions

Command	Data	Structure	WHERE allowed?	Family
DROP TABLE	all rows gone	gone	no	DDL
DELETE	matching rows gone	kept	yes	DML
TRUNCATE	all rows gone	kept	no	DDL
ALTER	kept	changed	no	DDL

```
SELECT   name, salary        -- 5
FROM     Employees           -- 1
WHERE    salary > 50000      -- 2
GROUP BY dept_id             -- 3
HAVING   COUNT(*) > 3        -- 4
ORDER BY salary DESC;        -- 6

INSERT INTO Students (id, name, dept) VALUES (1, 'Alice', 'CSE');
UPDATE Employees SET salary = salary * 1.10 WHERE dept = 'IT';
DELETE FROM Students WHERE id = 1;   -- WHERE makes DELETE selective
```



the order the clauses run in — not the order you write them

Exam trap **WHERE filters rows before grouping; HAVING filters groups after it.** That is the whole difference, and only HAVING may use an aggregate such as `HAVING COUNT(*) > 5`.

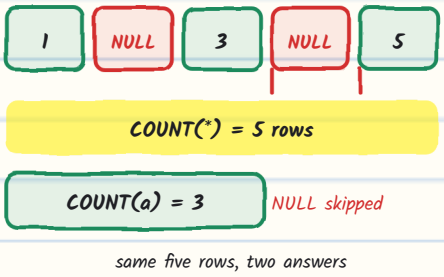
Memory trick + good to know **"Where before, having after."** *Where comes When rows are still rows.* `HAVING` without `GROUP BY` is legal too: the whole table is one group, so `SELECT COUNT(*) FROM Emp HAVING COUNT(*) > 5` returns either one number or nothing.

5 Aggregate Functions and the NULL Rules

every exam asks ★

Aggregate	Returns	NULL inside the column	Empty set
COUNT(*)	number of rows	counts them anyway	0
COUNT(col)	number of non-NULL values of col	skipped	0
SUM / AVG(col)	total / mean of the non-NULL values	skipped — never read as 0	NULL
MIN / MAX(col)	smallest / largest non-NULL value	skipped	NULL

↙ only COUNT(*) sees NULL



the same five rows, two counts

Worked example 1 ✍ COUNT T(a) holds 1, NULL, 3, NULL, 5.

```
SELECT COUNT(*), COUNT(a) FROM T;
```
→ (5, 3)

COUNT(*) counts rows, so the two NULL rows are counted; COUNT(a) counts values of a, and a NULL value is not a value.

Worked example 2 ✍ AVG R(score) holds 10, NULL, 20.

```
SELECT AVG(score) FROM R;
```
→ (10 + 20) / 2 = 15

Every aggregate except COUNT(*) ignores NULL; reading it as 0 would give 10 — the **wrong** answer.

Worked example 3 ✍ GROUP BY + HAVING

Sales(region, amount) = ('N',100), ('N',200), ('S',50), ('S',70), ('E',300)

```
SELECT region, SUM(amount) FROM Sales
GROUP BY region HAVING SUM(amount) > 150;
```

Sums: N = 300, S = 120, E = 300. Only N and E pass 150 → 2 rows.

Exam traps ✍ three of them **1.** AVG over 10, NULL, 20 is 15, not 10 — NULL is **unknown**, not zero. **2.** COUNT(*) on an empty table is 0, but AVG, SUM, MIN and MAX return NULL. **3.** A predicate on NULL is **unknown**, **never true**, so WHERE a > 0 drops those rows silently.

Memory trick "**Star sees every row; a column sees only real values.**" The asterisk stands for the whole tuple, so a row that exists is counted even when one of its values is missing. Read COUNT(*) as "count the rows", COUNT(col) as "count the answers".

6 The Six SQL Joins, Row by Row

count the rows ★

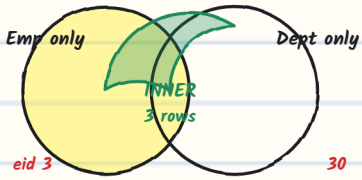
Emp(eid, dept_id)

eid	dept_id
1	10
2	20
3	NULL
4	10

Dept(dept_id, name)

dept_id	name
10	IT
20	HR
30	Finance

Join	Keeps	Rows
INNER	matches only	3
LEFT	all of Emp	4
RIGHT	all of Dept	4
FULL OUTER	all of both	5
CROSS	every pair	12
SELF	itself, aliased	rows & mgrs



FULL OUTER = 3 + 1 + 1 = 5

the three regions, and the row counts

INNER JOIN ✍ `SELECT e.eid, d.name FROM Emp e JOIN Dept d ON e.dept_id = d.dept_id;`
eid 1 → 10 IT · eid 2 → 20 HR · eid 4 → 10 IT. eid 3 has NULL, and **NULL = 10 is not true**, so it is dropped; dept 30 has no employee. → **3 rows**

LEFT and RIGHT JOIN ✍ **LEFT** = 3 matches + the unmatched employee padded → **4 rows**: (1,IT) (2,HR) (3,NULL) (4,IT). **RIGHT** = 3 matches + the unmatched department padded → **4 rows**: (1,IT) (2,HR) (4,IT) (NULL,Finance).

FULL OUTER and CROSS JOIN ✍ FULL OUTER = 3 matching pairs + 1 unmatched Emp row (eid 3) + 1 unmatched Dept row (30 Finance) = 3 + 1 + 1 = **5 rows**; the formula is **INNER + LEFT-only + RIGHT-only**. CROSS JOIN is 4 rows × 3 rows = **12 rows**, every row of one table against every row of the other with no condition at all; in general $m \times n$, so two 3-row tables give **9**.

SELF JOIN A relation joined to **itself** under two aliases, to relate rows inside one table — an employee to her manager. Give Emp two more columns, `name` and `mgr_id` (the manager's `eid`), then:
`SELECT a.name AS emp, b.name AS mgr FROM Emp a JOIN Emp b ON a.mgr_id = b.eid;`
Two aliases are **mandatory**: without them the condition would compare a column with itself.

Exam trap ✍ the NULL one An employee with a NULL `dept_id` is in **no** inner-join and **no** right-join result; it survives a **LEFT** and a **FULL OUTER** join, padded with NULLs. "Why did my row disappear?" → because **NULL never equals anything**, so the ON condition is unknown, not true.

Memory trick Read the keyword as a promise about the **first-named** table. **LEFT**: keep everything on the left, pad the right. **RIGHT**: swap the word. **FULL OUTER**: keep everything on both sides. **INNER**: keep only what agrees. **CROSS**: no promise at all.

7 Join Patterns, Subqueries and Where the Rest Lives

written pattern ★

Pattern: every department, including those with no employees The default join drops them, so ask for them with `LEFT JOIN` and count the **child** column, not *.

```
SELECT d.dept_name, COUNT(e.emp_id) AS employee_count
FROM   Department d
LEFT JOIN Employee e ON e.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name
ORDER BY employee_count DESC;
```

Exam traps 🔥 two counts, and WHERE that kills a LEFT JOIN `COUNT(e.emp_id)` skips the NULLs the outer join produced and correctly gives **0** for an empty department, while `COUNT(*)` counts the padded row and wrongly gives **1**; grouping by `dept_id` as well as `dept_name` keeps two departments that share a name apart. And a condition on the **right** table placed in `WHERE` is evaluated after the join and discards the padded rows, quietly turning your LEFT JOIN into an INNER JOIN — put it in `ON` instead.

Three kinds of subquery **Scalar** — one value used in a comparison: `WHERE salary > (SELECT AVG(salary) FROM Emp)`. **IN / EXISTS** — a set, or true/false: `WHERE id IN (SELECT emp_id FROM Emp)`. **Correlated** — references the outer row, so it is re-evaluated **once per outer row**: `WHERE e2.dept_id = e.dept_id`.

```
SELECT MAX(salary) AS second_highest        -- 2nd highest DISTINCT salary
FROM   Employee
WHERE  salary < (SELECT MAX(salary) FROM Employee);    -- NULL if there is none
-- and, above the average of the employee's OWN department:
SELECT e.name, e.salary, d.dept_name
FROM   Employee e JOIN Department d ON d.dept_id = e.dept_id
WHERE  e.salary > ( SELECT AVG(e2.salary) FROM Employee e2
                    WHERE e2.dept_id = e.dept_id );
```

Exam traps 🔥 three of them **1.** The correlated subquery runs **once per outer row**, giving e's own department average; a derived table pre-computing each average is usually the faster plan. **2.** The second-highest query uses <, not ≤, so it is right when the top salary repeats, and gives NULL when all salaries are equal. **3.** `NOT IN (subquery)` returns **no rows** if the subquery holds one NULL — prefer `NOT EXISTS`.

Where the rest of this chapter lives **Functional dependencies, INF–BCNF, normalisation → db3. File organisation, primary and secondary indexes, B+ trees, cost-based optimisation** — which index makes which algebra tree cheap → db4. **Transactions, ACID, isolation, concurrency, recovery → db5. Constraints and triggers** sit one line below this one on the syllabus and were covered with the data models — **db1**.

Night-Before-the-Exam Sheet

Fact sheet

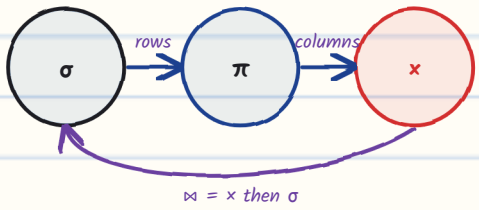
- σ = **rows** (SQL's WHERE)
- π = **columns** (SQL's SELECT list)
- ρ = rename · × = every row with every row
- Join = σ(R × S)
- DDL: CREATE, ALTER, DROP, TRUNCATE
- DML: SELECT, INSERT, UPDATE, DELETE
- DCL: GRANT, REVOKE
- TCL: COMMIT, ROLLBACK, SAVEPOINT
- DROP kills structure and data
- DELETE / TRUNCATE keep structure
- Run order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY
- WHERE filters rows **before** grouping
- HAVING filters groups **after** it
- COUNT(*) counts rows, NULLs included
- COUNT(col) counts non-NULL only
- AVG of 10, NULL, 20 = **15**
- Other aggregates skip NULL
- NULL never equals anything in a join
- FULL OUTER = INNER + left-only + right-only
- Six joins: INNER, LEFT, RIGHT, FULL, CROSS, SELF
- CROSS JOIN = m × n rows
- Zero-staff departments: `COUNT(e.emp_id)`

→ Top 12 one-liners for MCQs

1. Selection σ filters rows; projection π filters columns
2. Rename ρ renames; × pairs every row with every row
3. A join is a product plus a selection
4. DDL · DML · DCL · TCL — four families, four purposes
5. **DROP** removes structure and data together
6. **DELETE** and **TRUNCATE** both keep the structure
7. WHERE before grouping, HAVING after grouping
8. Only HAVING may use an aggregate
9. On 1, NULL, 3, NULL, 5: COUNT(*)=5, COUNT(a)=3
10. AVG of 10, NULL, 20 is 15, never 10
11. NULL never equals anything in a join condition
12. Zero-staff departments need `COUNT(e.emp_id)`

Top traps σ and π swapped · π removes duplicates but SELECT does not · DROP vs DELETE vs TRUNCATE · WHERE confused with HAVING · AVG reading NULL as 0 · expecting a NULL-department employee in an inner join · `COUNT(*)` for zero-employee counts · a WHERE on the right table turning a LEFT JOIN into an INNER JOIN.

All tricks together Sigma picks rows, pi picks columns · product first, then pick · where before, having after · star sees every row, a column sees only real values · keep everything on the left, pad the right · join by an alias, never by the bare table name.



two primitives make every join

✓ Practice MCQs (exam pattern, 10 marks)

try first, then check

1. Which SQL command removes a table's structure together with all its data?
(a) DELETE (b) TRUNCATE
(c) ALTER (d) DROP

2. Which clause filters groups after `GROUP BY` has been applied?
(a) HAVING (b) WHERE
(c) ORDER BY (d) DISTINCT

3. Which relational algebra operator keeps the tuples that satisfy a condition?
(a) Projection (π) (b) Rename (ρ)
(c) Selection (σ) (d) Cartesian product (×)

4. T has one column a with rows 1, NULL, 3, NULL, 5. `SELECT COUNT(*), COUNT(a) FROM T;` gives:
(a) 3 and 3 (b) 5 and 3
(c) 5 and 5 (d) 3 and 5

5. `R.score` holds 10, NULL, 20. `SELECT AVG(score) FROM R;` returns:
(a) 10 (b) 15
(c) 20 (d) NULL

6. With `Emp` = (1,10),(2,20),(3,NULL),(4,10) and `Dept` = (10,'IT'),(20,'HR'),(30,'Finance'), the INNER JOIN on `dept_id` returns:
(a) 3 rows (b) 4 rows
(c) 5 rows (d) 12 rows

7. The same data with a FULL OUTER JOIN returns:
(a) 3 rows (b) 4 rows
(c) 5 rows (d) 6 rows

8. `Sales` = ('N',100),('N',200),('S',50),('S',70),('E',300). `GROUP BY region HAVING SUM(amount) > 150` returns:
(a) 1 row (b) 3 rows
(c) 5 rows (d) 2 rows

9. Listing every department with its employee count, including empty ones, needs:
(a) COUNT(*) (b) COUNT(e.emp_id)
(c) COUNT(d.dept_name) (d) SUM(e.salary)

10. In SQL, relational algebra's selection σ corresponds to:
(a) the WHERE clause (b) the SELECT list
(c) GROUP BY (d) ORDER BY

Answer key → 1-d · 2-a · 3-c · 4-b · 5-b · 6-a · 7-c · 8-d · 9-b · 10-a

Written question Explain the fundamental operations of relational algebra on `Students(id, name, dept)` and `Courses(cid, name)`. Distinguish DDL, DML, DCL and TCL, and say exactly what DROP, DELETE, TRUNCATE and ALTER each do. Then, for `Emp` = (1,10),(2,20),(3,NULL),(4,10) and `Dept` = (10,'IT'),(20,'HR'),(30,'Finance'), give the row count of INNER, LEFT, RIGHT, FULL OUTER and CROSS JOIN and explain why employee 3 never appears in an inner join. 10 marks

1. **RA operators (2)** — **σ** keeps rows meeting a condition, **π** keeps columns and removes duplicates, **ρ** renames, **×** pairs every row of one relation with every row of another (2 × 2 = **4**), **∪ − ∩** are the set operators, and **⋈** is derived as σ(R × S). 2
2. **Command families (2)** — **DDL** (Definition) on **structure**: CREATE, ALTER, DROP, TRUNCATE. **DML** (Manipulation) on **rows**: SELECT, INSERT, UPDATE, DELETE. **DCL** (Control) on **privileges**: GRANT, REVOKE. **TCL** (Transaction Control): COMMIT, ROLLBACK, SAVEPOINT. 2
3. **The four commands (2)** — **DROP** removes structure and data; **DELETE** removes rows, optionally by WHERE, keeping the structure; **TRUNCATE** removes all rows, keeps the structure, allows no WHERE; **ALTER** changes the structure and keeps the data. 2
4. **Row counts (1)** — INNER **3**, LEFT **4**, RIGHT **4**, FULL OUTER **5** = 3 + 1 + 1, CROSS 4 × 3 = **12**. 1
5. **Why employee 3 disappears (2)** — its `dept_id` is NULL and **NULL = 10 is unknown, not true**; an inner join keeps a row only when the ON condition is **true**. It survives LEFT and FULL OUTER, padded with NULLs. 2
6. **Where vs having (1)** — WHERE filters **rows before** grouping; HAVING filters **groups after** `GROUP BY`, and is the only one that may use an aggregate. 1

→ Before the exam ✓

- ✓ Can name σ, π, ρ, × with an example each
- ✓ Can write a join as σ(R × S) and explain ρ
- ✓ Can place any command in DDL, DML, DCL or TCL
- ✓ Can say what DROP, DELETE, TRUNCATE, ALTER do
- ✓ Can count the rows of all six joins
- ✓ Can state the COUNT(*) / AVG-with-NULL rules